

Business Domain: Retail

Business: Electronics stores

Business problem: The electronics retail store lacks a centralized, data-driven view of its overall business performance. Management faces difficulties in understanding sales trends, customer purchasing behavior, product profitability, store performance, and employee productivity. This makes it challenging to identify areas of strong performance, detect underperforming areas, and make informed decisions to improve sales and profitability.

Business Requirements & Questions

1. Analyze Sales Performance:

- What are the total sales and how do they change over time?
- 2. Which periods generate the highest and lowest sales?

2. Customer Analysis:

- 1. Who are the highest-value customers based on total spending?
- 2. Which customer segments contribute the most to sales?

3. Product Performance & Profitability

- Which products generate the highest sales and quantities sold?
- 2. Which products and categories are the most profitable?

4. Store Performance:

- 1. Which stores generate the highest sales and profit?
- 2. Which stores are underperforming compared with others?

5. Employee Performance:

- 1. Which employees generate the highest sales?
- 2. Which employees contribute the most to profit?

6. Business Improvement:

- 1. Which products or stores have high sales but low profitability?
- 2. What are the main opportunities for improving sales and profitability?

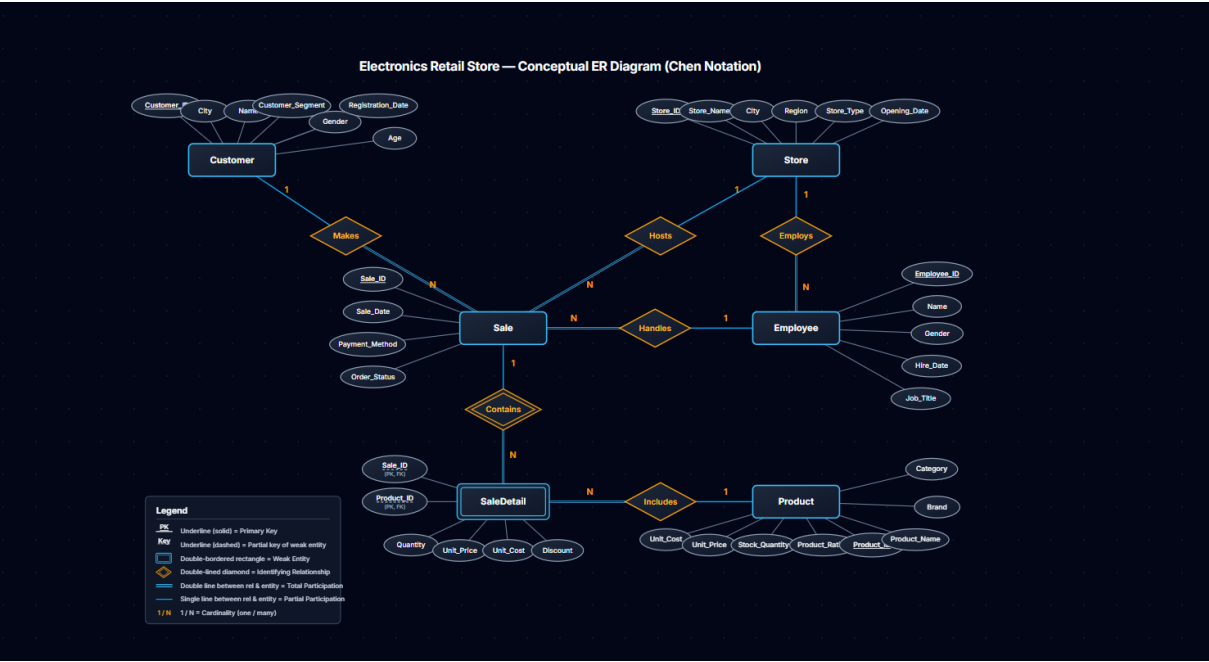
KPIs

- Total Sales
- Total Profit
- Profit Margin %
- Total Customers
- Total Quantity Sold

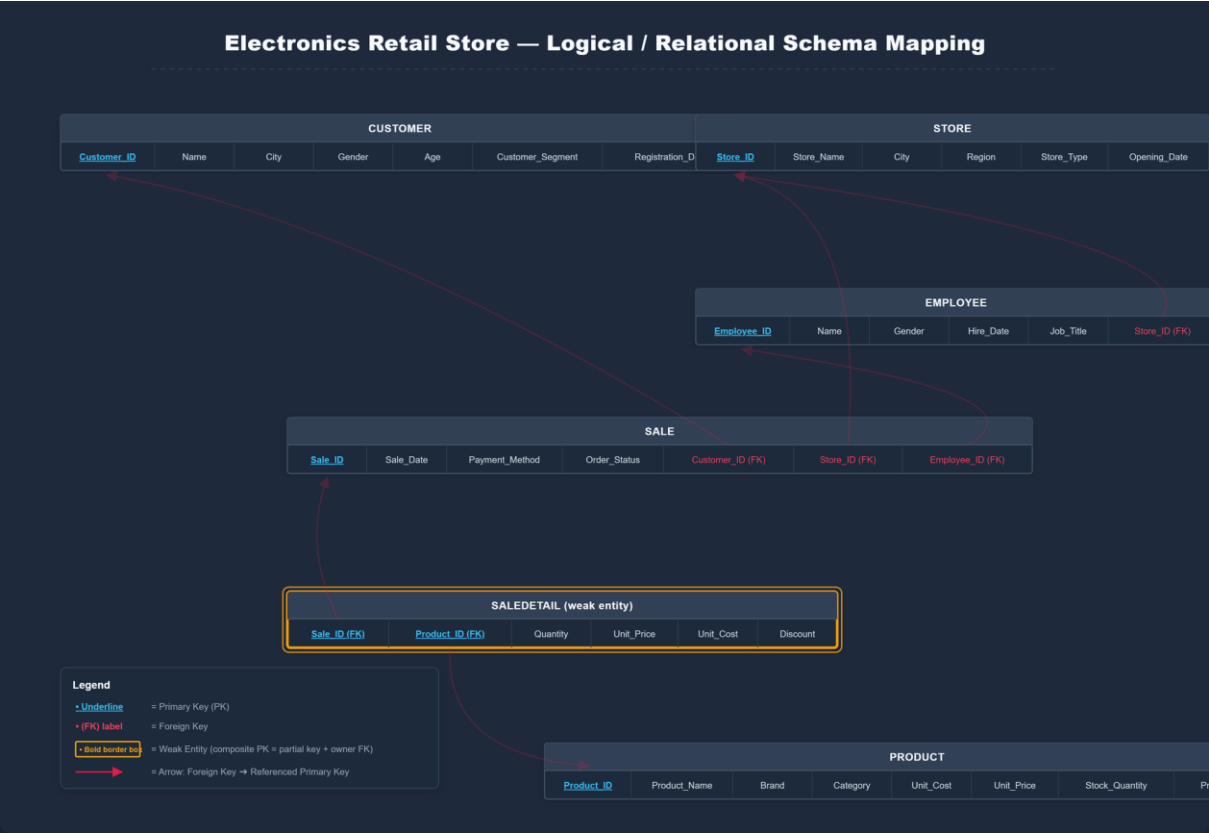
The Scenario

- The company is an electronics retail store that has a number of customers. Each customer has a unique Customer_ID, Name, Gender, Age, City, Customer_Segment, and Registration_Date.
- The company sells a variety of electronic products. Each product has a unique Product_ID, Product_Name, Brand, Category, Unit_Cost, Unit_Price, Stock_Quantity, and Product_Rating.
- The company operates several stores in different cities and regions. Each store has a unique Store_ID, Store_Name, City, Region, Store_Type, and Opening_Date.
- Each store has a number of employees, and each employee works in only one store. Each employee has a unique Employee_ID, Name, Gender, Hire_Date, and Job_Title.
- Customers make sales transactions through the company's stores. Each sale has a unique Sale_ID, Sale_Date, Payment_Method, and Order_Status. Each sale is associated with one customer, one store, and one employee.
- Each sale contains one or more products, and a product may be included in many different sales. For each product included in a sale, the company records the Quantity, Unit_Price at the time of sale, Unit_Cost at the time of sale, and Discount. Each SaleDetail is uniquely identified by the combination of Sale_ID and Product_ID, which together form a composite key for the SaleDetail entity.
- Each sale is handled by one employee, while an employee may handle many sales.
- Each employee works at one store, while a store may have several employees and may have no sales transactions during a particular period.

ERD



Mapping



Data Dictionary

1. Customers

- **CustomerID:** Unique identifier for each customer. **(PK)**
- **CustomerName:** Full name of the customer.
- **Gender:** Gender of the customer.
- **Age:** Age of the customer.
- **City:** City where the customer lives.
- **CustomerSegment:** Customer classification based on purchasing behavior or characteristics.
- **RegistrationDate:** Date when the customer registered with the store.

2. Products

- **ProductID:** Unique identifier for each product. **(PK)**
- **ProductName:** Name of the electronic product.
- **Brand:** Brand or manufacturer of the product.
- **Category:** Category of the product, such as smartphones, laptops, or accessories.
- **UnitCost:** Cost incurred by the business to acquire one unit of the product.
- **UnitPrice:** Selling price of one unit of the product.
- **StockQuantity:** Current quantity of the product available in stock.
- **ProductRating:** Customer rating of the product.

3. Stores

- **StoreID:** Unique identifier for each store or branch. **(PK)**
- **StoreName:** Name of the store or branch.
- **City:** City where the store is located.
- **NumberOfEmployees:** Number of employees working at the store.
- **OpeningDate:** Date when the store was opened.

4. Employees

- **EmployeeID:** Unique identifier for each employee. **(PK)**
- **EmployeeName:** Full name of the employee.
- **Gender:** Gender of the employee.
- **HireDate:** Date when the employee was hired.
- **JobTitle:** Job position or role of the employee.
- **StoreID:** Identifier of the store where the employee works. **(FK)**

5. Sales

- **SaleID:** Unique identifier for each sales transaction. **(PK)**
- **SaleDate:** Date when the sale transaction occurred.
- **CustomerID:** Identifier of the customer who made the purchase. **(FK)**
- **StoreID:** Identifier of the store where the sale took place. **(FK)**
- **EmployeeID:** Identifier of the employee responsible for the sale. **(FK)**

- **PaymentMethod:** Method used by the customer to pay for the purchase.
- **OrderStatus:** Current status of the sales transaction.

6. SaleDetails

- **SaleID:** Identifier of the related sales transaction. **(PK, FK)**
- **ProductID:** Identifier of the product included in the sale. **(PK, FK)**
- **Quantity:** Number of units of the product sold.
- **UnitPrice:** Selling price per unit at the time of the transaction.
- **UnitCost:** Cost per unit at the time of the transaction.
- **Discount:** Discount applied to the product, represented as a decimal value less than 1.

How I get the data?

First, I focused on making the dataset as realistic as possible in terms of the sales period and sales volume. The sales data represents approximately six months of transactions, with the dataset size set as follows:

- SaleDetails: 3,000 rows
- Sales: 1,200 rows
- Products: 120 rows
- Customers: 350 rows
- Employees: 18 rows
- Stores: 3 rows

How I get the messy data?

I provided all the data cleaning tools we learned, and based on them, I intentionally created the missing and dirty data.

I distributed different types of missing and inconsistent data across all tables. Each table will be used as a sample for a specific cleaning tool, allowing me to demonstrate and apply all the tools available within each tool throughout the data cleaning process.

I used the tools as follows:

- MySQL → Sales
- Power Query → Customers and Products
- Python (Pandas) → SaleDetails

Data Cleaning Documentation, Phase 3

0. Database Setup

- Received the database from Mohamed.
- Created the database locally.
- Verified that the database and its tables were available for the cleaning process.

1. SQL Cleaning

1.1 Sales Table

Profiling

- Create backup copy: sales_raw.
- Calculate total rows.
- Check Data Types and Constraints.
- Check NULL and blank values.
- Check duplicate sale_id.
- Check distinct payment_method values.
- Check distinct order_status values.
- Validate foreign-key relationships with customers, stores, and employees.

Payment Method Cleaning

- Preview old and new payment_method values using CASE.
- Convert variations to Credit Card, Debit Card, Cash, Mobile Wallet, and Installment.
- Convert unknown or NULL values to Unknown.
- Apply changes using UPDATE.
- Check distinct payment_method values after cleaning.

Order Status Cleaning

- Preview old and new order_status values using CASE.
- Standardize order_status values.
- Convert variations to Pending, Completed, Cancelled, and Returned.
- Convert unknown or NULL values to Unknown.
- Apply changes using UPDATE.
- Check distinct order_status values after cleaning.

Validation

- Recheck total rows.
- Recheck NULL and blank values.
- Recheck distinct payment_method values.
- Recheck distinct order_status values.

1.2 Stores Table

Profiling

- Create backup copy: stores_raw.
- Check Data Types and Constraints.
- Calculate total rows.
- Check NULL and blank values.
- Check duplicate store_id.
- Check distinct city values.
- Validate employee-to-store relationships.

Result: No issues requiring cleaning were found, so no modifications were performed.

2. Power Query Cleaning

2.1 Customers Table

Name

- Trim.
- Lowercase.
- Capitalize Each Word.

Gender

- Uppercase.
- Trim.
- Replace Value: Blank → null.
- Replace Value: NULL → null.
- Custom Column: Standardize Gender Values.

```
if [gender] = "F" or [gender] = "FEMALE" then "Female"
else if [gender] = "M" or [gender] = "MALE" then "Male"
else if [gender] = null then
    if [name] = "Farouk Youssef" then "Male"
    else if [name] = "Sanaa Hany" then "Female"
    else if [name] = "Karim Osama" then "Male"
    else if [name] = "Sherif Youssef" then "Male"
    else if [name] = "Sherif Awad" then "Male"
    else null
else [gender]
```

- Convert F / FEMALE → Female.
- Convert M / MALE → Male.
- Unknown Gender (based on name):
 - Male: (Farouk Youssef, Karim Osama, Sherif Youssef, Sherif Awad) → Male.
 - Female: (Sanaa Hany) → Female.
- Remove the original "gender" column.

Age

- Replace Value: NULL → null.

- Changed type text → whole number
- Custom Column.

```
if [age] = null or [age] > 60 then
    List.Average(List.RemoveNulls("#Changed Type1"[age]))
else [age]
```

- Age > 60 or NULL → replaced with the average age; otherwise, keep the original age.
- Changed type text → whole number
- Remove the original "age" column.
- Replace NULL values.
- Replace ages greater than 60 with average age.
- Round average to whole number.

City

- Trim
- Capitalize Each Word
- Replace Values: Blank → Unknown.

2.2 Products Table

Product Name

- Trim.
- Capitalize Each Word.

Brand

- Replace Values: anker → Anker; DELL → Dell.

Category

- Trim
- Custom Column to standardize category values.

```
if [category] = "HEADPHONES" or [category] = "headphones" then "Headphones"
else if [category] = "LAPTOPS" then "Laptops"
else if [category] = "monitors" then "Monitors"
else [category]
```

- HEADPHONES / headphones → Headphones.
- LAPTOPS → Laptops.
- monitors → Monitors.
- Remove the original "category" column.

3. Python Cleaning

3.1 Sale Details Table

Profiling

- Import Pandas library using `import pandas as pd`.
- Load `sale_details.csv` using Pandas.
- Preview the first rows using `df.head()`.
- Check total rows and columns using `df.shape`.
- Check data types and non-null counts using `df.info()`.
- Check NULL values using `df.isnull().sum()`.
- Check duplicate rows using `df.duplicated().sum()`.
- Generate statistical summary using `df.describe()`.

Cleaning

- Check records where quantity is less than or equal to 0.
- Remove records where quantity is less than or equal to 0.
- Check records where `unit_cost` is greater than or equal to `unit_price`.
- Load the products reference file: `products PQ.xlsx`.
- Merge `sale_details` with `products` using `product_id`.
- Compare `sale unit_price` and `unit_cost` with the corresponding product reference values.
- Replace `unit_price` and `unit_cost` with the values from the products reference table.
- Remove temporary sale price and cost columns after correction.
- Recheck `unit_cost` and `unit_price` to confirm that `unit_cost` is lower than `unit_price`.
- Check discount values outside the valid range of 0 to 1.
- Convert invalid discount values to NULL (`pd.NA`).
- Replace missing discount values with the mean discount.

Validation

- Recheck discount values to confirm that no values are below 0 or above 1.
- Recheck NULL values across all columns.
- Check the final dataset size.
- Recheck quantity to confirm that no value is less than or equal to 0.

Final Result

- 20 invalid quantity records were removed.
- Incorrect `unit_price` and `unit_cost` values were corrected using the products reference table.
- 48 invalid/missing discount values were handled and replaced using the mean discount.
- No NULL values remained after cleaning.
- No invalid discount values remained after validation.
- Final dataset size: 980 rows and 6 columns.

Export

- Export the cleaned dataset as `sale_details_cleaned.csv` for re-import into SQL.

Phase 4 — SQL Summarization

1. Data Re-importing

Re-imported the cleaned data into the SQL database and verified that the required tables and relationships were available and working properly.

2. Data Joining

Created SQL queries using JOIN to connect related tables such as Products, Sales, SaleDetails, Stores, Employees, and Customers.

3. Data Aggregation

Used Aggregate Functions such as SUM(), COUNT(), and AVG() with GROUP BY to perform data summarization.

4. Key Metrics Calculation

Calculated the main Key Metrics, including Total Quantity, Total Orders, Total Revenue, Total Cost, and Total Profit.

5. Performance Summaries

Created Performance Summaries for Products, Categories, Brands, Stores, Employees, and Customers.

6. Subquery Analysis

Used a Subquery with AVG() and HAVING to identify products whose sales quantity was above the average.

7. Final SQL Summaries

Prepared the resulting SQL summaries to support the following phases, especially Analytical Modeling and Data Visualization.

Phase 5: By Ahmad Mohamed Saad El-Din Mohamed

NTI Data Analytics Track | End-to-End Business Intelligence & Reporting

Project Overview & Executive Summary

- Program: National Telecommunication Institute (NTI) - Data Analytics & BI Track
- Project Title: Smart Zone Retail Analytics & Performance Intelligence
- Tools & Technologies: Excel Power Pivot, DAX (Data Analysis Expressions), Star Schema Modeling, Data Storytelling
- Key Metric Highlights: Total Revenue: 106.40M EGP | Total Profit: 12.07M EGP | Overall Margin: 11.34% | Transactions: 1,200

1. Project Background & Business Objectives

Smart Zone is a leading retail electronics enterprise operating multiple branches across key Egyptian governorates (Cairo, Alexandria, and Mansoura). The primary business mandate of this project was to transition from scattered transactional logs into a centralized, scalable Business Intelligence (BI) data model to evaluate financial health, regional profitability, workforce productivity, and payment ecosystem trends.

- **Revenue & Profitability Assessment:** Quantify total top-line sales and bottom-line margin percentages across stores and departments.
- **Operational Efficiency:** Evaluate store-level performance and uncover factors driving regional variance in profit margins.
- **Workforce Analytics:** Identify top-performing sales representatives versus low-activity personnel to optimize incentives and training.
- **Product Mix & Payment Preferences:** Analyze volume vs. revenue across hardware categories and assess customer reliance on credit, cash, and installment options.

2. Data Architecture & Dimensional Modeling (Star Schema)

To ensure optimal query performance and intuitive analytical slicing in Power Pivot, the data was modeled into a standard Star Schema. The central Fact table captures granular retail events, linked to Dimension tables via one-to-many (1:*) relationships.

Table Name	Table Type	Primary / Foreign Key	Key Attributes & Role
Fact_Sales	Fact Table	FK: store_id, product_id, emp_id, date	Transactional records (1,200 rows), quantity sold, payment method, unit financials.
Dim_Stores	Dimension	PK: store_id	Branch metadata, store name (Cairo, Alex, Mansoura), location details.
Dim_Products	Dimension	PK: product_id	Product category (Laptops, Smartphones, etc.), unit cost, selling price.
Dim_Employees	Dimension	PK: emp_id	Sales representative details, branch

assignment, transaction volume.

3. Power Pivot & DAX Measures Implementation

Key Performance Indicators (KPIs) and analytical metrics were computed inside Power Pivot using Data Analysis Expressions (DAX). The formulas guarantee dynamic recalculation across all pivot dimensions and slicers:

- **Total Revenue:** Calculates gross sales value across all completed transactions.
Formula: $\text{Total Revenue} = \text{SUMX}(\text{Fact_Sales}, \text{Fact_Sales}[\text{quantity}] * \text{RELATED}(\text{Dim_Products}[\text{price}]))$
- **Total Cost:** Computes total Cost of Goods Sold (COGS) based on product baseline cost.
Formula: $\text{Total Cost} = \text{SUMX}(\text{Fact_Sales}, \text{Fact_Sales}[\text{quantity}] * \text{RELATED}(\text{Dim_Products}[\text{cost}]))$
- **Total Profit:** Net operational gross margin generated before OPEX overhead.
Formula: $\text{Total Profit} = [\text{Total Revenue}] - [\text{Total Cost}]$
- **Profit Margin %:** Evaluates percentage return per EGP of revenue generated.
Formula: $\text{Profit Margin \%} = \text{DIVIDE}([\text{Total Profit}], [\text{Total Revenue}], 0)$
- **Total Quantity Sold:** Total units of physical hardware and accessories sold (7,308 units).
Formula: $\text{Total Quantity Sold} = \text{SUM}(\text{Fact_Sales}[\text{quantity}])$
- **Total Transactions:** Total individual checkout receipts recorded (1,200 transactions).
Formula: $\text{Total Transactions} = \text{COUNTROWS}(\text{Fact_Sales})$
- **Profit Margin KPI Status:** Threshold indicator flagging branches achieving healthy double-digit margins.
Formula: $\text{Profit Margin \% Status} = \text{IF}([\text{Profit Margin \%}] \geq 0.10, 1, 0)$

4. Comprehensive Analysis & Executive Findings

4.1 Branch Financials & Profitability Analysis

Analysis of the three operational branches reveals an important commercial paradox: While Smart Zone Cairo drives the highest gross revenue, Smart Zone Mansoura delivers the highest absolute profit and the highest profit margin.

Store Name	Total Revenue (EGP)	Total Profit (EGP)	Profit Margin %	KPI Status
Smart Zone Cairo	37,758,146.69	4,068,750.49	10.78%	On Target (1)
Smart Zone Alexandria	36,411,294.59	3,685,437.67	10.12%	On Target (1)
Smart Zone Mansoura	32,230,519.59	4,313,860.27	13.38%	Top Margin (1)
Grand Total	106,399,960.87	12,068,048.43	11.34%	Overall Healthy

4.2 Sales Performance by Product Category

Category	Units Sold	Total Revenue (EGP)	% of Total Revenue
Laptops	1,409	44,939,874.56	42.24%
Smartphones	1,172	27,335,631.29	25.69%
Tablets	1,001	17,809,826.91	16.74%

Headphones	1,076	7,090,520.60	6.66%
Monitors	762	6,293,044.06	5.91%
Accessories	1,104	1,552,411.00	1.46%
Keyboards & Mice	784	1,378,652.45	1.30%
Grand Total	7,308	106,399,960.87	100.00%

4.3 Payment Method Dynamics & Workforce Productivity

- **Installment & Credit Dominance:** Installment plans generated the single highest revenue stream (22.89M EGP across 211 transactions), followed closely by Credit Cards (22.01M EGP across 232 transactions). Combined, deferred credit mechanisms account for over 42% of company turnover.
- **Mobile Wallets as Volume Driver:** Mobile Wallets captured the highest transaction count (259 transactions, generating 20.16M EGP), proving high consumer adoption in Egyptian retail.
- **Data Quality Anomaly:** The presence of 41 'Unknown' payment transactions (worth 5.45M EGP) represents a POS data validation gap requiring immediate system-level enforcement.
- **Sales Workforce Distribution:** Ibrahim Ashraf led the organization in revenue generation (12.93M EGP, 112 transactions), closely followed by Tamer Ehab (12.50M EGP, 79 transactions). Shorouk Youssef registered the highest transaction count (122 transactions, 7.28M EGP). In contrast, low-frequency personnel (e.g., Magdy Zidan with 31 transactions / 707K EGP) highlight potential staffing or territory imbalances.

5. Strategic Recommendations & Action Plan

- **1. Replicate Mansoura's High-Margin Strategy:** Analyze Mansoura's exact product bundle mix and sales techniques to elevate Alexandria (10.12%) and Cairo (10.78%) margins toward the 13.38% benchmark.
- **2. Expand High-Margin Accessories Attachment:** Accessories and Keyboards/Mice currently represent under 3% of sales despite high unit volume. Implement mandatory cross-selling prompts at checkout (e.g., laptop sleeves, screen protectors, wireless mice).
- **3. Formalize BNPL & Fintech Partnerships:** Given that installment and mobile wallet payments drive over 43M EGP in revenue, partner directly with BNPL providers (ValU, Sympl, Aman) to reduce transaction fees and offer zero-interest promotions.
- **4. Workforce Incentive Restructuring:** Institute tiered commission structures rewarding both revenue volume (benchmarked by Ibrahim Ashraf and Tamer Ehab) and high-margin product push.
- **5. POS Data Governance:** Mandate mandatory payment method selection at point-of-sale systems to eliminate 'Unknown' payment records.

Documentation by Reem Mahmoud — Phase 6

Full Documentation of the Code — Python

Step 1: Importing Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files
```

- **pandas**: Used for reading Excel files and handling tabular data.
- **matplotlib.pyplot**: Provides basic plotting functions.
- **seaborn**: A visualization library built on top of matplotlib, used for more attractive statistical plots.
- **files**: Allows uploading files into Google Colab.

Step 2: Uploading Files

```
uploaded = files.upload()
```

- This command opens a file upload dialog in Google Colab.
- You are expected to upload six files:
 - customers PQ.xlsx
 - products PQ.xlsx
 - employees PQ.xlsx
 - sale_details_cleaned-1.xlsx
 - sales2.xlsx
 - Stores SQL.xlsx

Step 3: Loading Datasets

```
customers = pd.read_excel("customers PQ.xlsx")
products = pd.read_excel("products PQ.xlsx")
employees = pd.read_excel("employees PQ.xlsx")
sale_details = pd.read_excel("sale_details_cleaned-1.xlsx")
sales = pd.read_excel("sales2.xlsx")
stores = pd.read_excel("Stores SQL.xlsx")
```

- Each Excel file is loaded into a separate **DataFrame**.
- These DataFrames represent different aspects of the business: customers, products, employees, detailed sales transactions, sales orders, and store information.

◇ Customers Insights

- **Printed Metrics**:
 - Total number of customers.
 - Average age of customers.
 - Number of unique cities customers belong to.

- **Visualizations:**
 - **Bar Chart:** Distribution of customers per city.
 - **Histogram:** Age distribution of customers.

◇ Products Insights

- **Printed Metrics:**
 - Total number of products.
 - Product with the highest rating.
 - Product with the lowest price.
- **Additional Calculation:**
 - Profit per product = unit_price - unit_cost.
- **Visualizations:**
 - **Bar Chart:** Top 10 products by rating.
 - **Bar Chart:** Top 10 products by profit.

◇ Employees Insights

- **Printed Metrics:**
 - Total number of employees.
 - Number of unique job titles.
 - Number of stores covered.
- **Visualizations:**
 - **Bar Chart:** Employees per job title.
 - **Bar Chart:** Employees per store.

◇ Sale Details Insights

- **Printed Metrics:**
 - Total number of sale records.
 - Average discount applied.
 - Total profit across all sales.
 - Product ID with the highest total quantity sold.
- **Visualizations:**
 - **Histogram:** Distribution of discounts.
 - **Bar Chart:** Top 5 products by quantity sold.

◇ Sales Insights

- **Printed Metrics:**
 - Total number of sales orders.
 - Number of completed, cancelled, and returned orders.
 - Number of unique payment methods used.
- **Visualizations:**
 - **Pie Chart:** Distribution of order statuses.

- **Bar Chart:** Orders by payment method.

◇ Stores Insights

- **Printed Metrics:**
 - Total number of stores.
 - Number of unique cities covered.
 - Earliest store opening date.
 - Latest store opening date.
- **Visualizations:**
 - **Bar Chart:** Stores per city.

◇ Linear Regression Model

```
from sklearn.linear_model import LinearRegression

X = products[["unit_cost"]] # Feature: product cost
y = products[["unit_price"]] # Target: product price
model = LinearRegression().fit(X, y)
print("Predicted Price:", model.predict([[1000]])[0])
```

- A **Linear Regression model** is trained to predict product prices based on their costs.
- The model is tested by predicting the price of a product with a cost of 1000.
- This demonstrates a simple machine learning application integrated into the analysis.

✦ Summary

This code performs:

1. **Data Upload & Preparation** — Reading six Excel datasets.
2. **Summarized Insights** — Printing key metrics for customers, products, employees, sales details, sales orders, and stores.
3. **Visualization Insights** — Creating charts (bar, histogram, pie) to visually summarize the data.
4. **Predictive Modeling** — Using Linear Regression to estimate product prices based on cost.

Documentation of the Dashboard

Executive Summary

Objective: Provides executive leadership with macro-level insights into financial health, profit margins, sales trajectories, and payment method distribution.

- **KPI Cards:** Total Sales, Total Profit, Profit Margin %, Total Orders
- **Line Chart (Sales & Profit Trend):** X-axis: sale_date (sales) | Y-axis: Total Sales, Total Profit
- **Clustered Bar Chart (Revenue by Store):** Y-axis: store_name (Stores SQL) | X-axis: Total Sales
- **Donut Chart (Payment Method Share):** Legend: payment_method (sales) | Values: Total Sales
- **Slicers:** Date Range (sale_date), Store Location (city)

Product & Category Analysis

Objective: Examines product portfolio performance, item-level margins, top-grossing items, and discount impacts.

- **KPI Cards:** Total Sales, Total Quantity Sold, Total Discount Amount
- **Clustered Bar Chart (Top 10 Products):** Y-axis: product_name (products PQ filtered via Top N = 10 by Total Sales) | X-axis: Total Sales
- **Clustered Column Chart (Category Performance):** X-axis: category (products PQ) | Y-axis: Total Sales, Total Profit
- **Treemap (Brand Share):** Category: brand (products PQ) | Values: Total Sales
- **Table Visual (Product Matrix):** product_name, category, brand, Total Quantity Sold, Total Sales, Total Profit, product_rating
- **Slicers:** Category (category), Brand (brand)

Customers & Stores Analysis

Objective: Analyzes regional customer penetration, age/gender purchasing habits, and store city footprints.

- **KPI Cards:** Total Customers, Average Spend per Customer, Average Customer Age
- **Map Visual (Geographical Sales Map):** Location: city (customers PQ, Data Category = City) | Bubble size: Total Sales
- **Donut Chart (Gender Revenue Split):** Legend: gender (customers PQ) | Values: Total Sales
- **Clustered Column Chart (Top 10 High-Value Customers):** X-axis: name (customers PQ filtered via Top N = 10 by Total Sales) | Y-axis: Total Sales
- **Clustered Bar Chart (Store Performance by City):** Y-axis: city (Stores SQL) | X-axis: Total Sales
- **Slicers:** Store Location (city), Customer Gender (gender)

Employee Performance

Objective: Measures sales force productivity, order throughput per employee, and revenue contribution by job title.

- **KPI Cards:** Total Sales, Total Employees, Average Sales per Employee
- **Clustered Bar Chart (Individual Staff Sales):** Y-axis: name (employees PQ) | X-axis: Total Sales
- **Clustered Column Chart (Order Volume by Employee):** X-axis: name (employees PQ) | Y-axis: Total Orders
- **Donut Chart (Revenue by Job Title):** Legend: job_title (employees PQ) | Values: Total Sales
- **Table Visual (Employee Performance Scorecard):** name, job_title, store_id, Total Orders, Total Sales, Total Profit
- **Slicers:** Job Role (job_title), Gender (gender)

Financial & Sales DAX

```
Total Sales = SUMX('sale_details_cleaned',  
    'sale_details_cleaned'[quantity] * 'sale_details_cleaned'[unit_price] * (1 -  
    'sale_details_cleaned'[discount]))
```

```
Total Cost = SUMX('sale_details_cleaned',  
    'sale_details_cleaned'[quantity] * 'sale_details_cleaned'[unit_cost])
```

```
Total Profit = [Total Sales] - [Total Cost]
```

```
Profit Margin % = DIVIDE([Total Profit], [Total Sales], 0)
```

Volume & Discount DAX

```
Total Orders = DISTINCTCOUNT('sales'[sale_id])
```

```
Total Quantity Sold = SUM('sale_details_cleaned'[quantity])
```

```
Total Discount Amount = SUMX('sale_details_cleaned',  
    'sale_details_cleaned'[quantity] * 'sale_details_cleaned'[unit_price] *  
    'sale_details_cleaned'[discount])
```

Demographics & Staff DAX

```
Total Customers = DISTINCTCOUNT('customers PQ'[customer_id])
```

```
Average Spend per Customer = DIVIDE([Total Sales], [Total Customers], 0)
```

```
Average Customer Age = AVERAGE('customers PQ'[age])
```

```
Total Employees = DISTINCTCOUNT('employees PQ'[employee_id])
```

```
Average Sales per Employee = DIVIDE([Total Sales], [Total Employees], 0)
```